

SOFTWARE DESIGN DOCUMENT (SDD)

Utility Billing System

1. DOCUMENT CONTROL

Item	Details
Project Name	Utility Billing System
Version	1.0
Author	Yaswitha
Date	29 December, 2025
Status	Final

2. PURPOSE OF THE DOCUMENT

This document describes the system architecture, design decisions, component structure, APIs, data models, and non-functional aspects of the **Utility Billing System**.

It is intended for:

- Developers
 - Reviewers
 - Interview discussions
 - Maintenance & enhancement planning
-

3. SYSTEM OVERVIEW

Business Objective

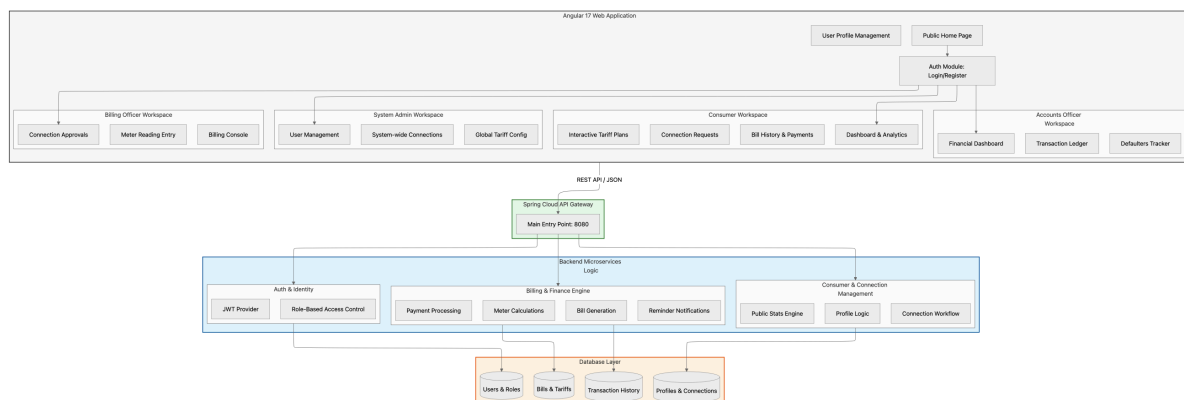
Provide a scalable, secure, and maintainable microservices-based system to:

- Manage consumer profiles and utility connections (Electricity, Water, Gas, Internet).
- Automate meter reading inputs and bill generation based on configurable tariffs.
- Streamline payment tracking and user notifications.
- Support role-based operations for Admins, Billing Officers, and Consumers.

High-Level Features

- **User Management:** Registration, Login, Role Management (Admin, Consumer, Billing Officer).
- **Consumer Operations:** Profile creation, Connection requests (linking Meters to Users).
- **Utility & Tariff Config:** Admin-managed utility types and tiered pricing slabs.
- **Meter Management:** Recording monthly readings and calculating consumption.
- **Billing Engine:** Auto-calculation of bills based on consumption and tariff rates.
- **Notifications:** Async email alerts for welcome messages and bill generation.

4. ARCHITECTURE OVERVIEW (HLD)



Architecture Style

- **Microservices Architecture**
- **REST-based communication** (Synchronous)
- **Event-Driven communication** (Asynchronous via Kafka)
- **Containerized deployment** (Docker)

Core Components

- **Angular Frontend:** Single Page Application (SPA) for all user roles.
- **API Gateway:** Single entry point for routing and security.

- **Discovery Server:** Netflix Eureka for service registration.
 - **Config Server:** Centralized configuration management.
 - **Domain Services:** Auth, Consumer, Utility, Meter, Billing, Notification.
 - **Infrastructure:** MongoDB (per service), Kafka (messaging), Zookeeper.
-

5. TECHNOLOGY STACK

BACKEND

Language: Java 17

Framework: Spring Boot

Reactive Stack: Spring WebFlux (Project Reactor)

Database Access: Spring Data MongoDB (Reactive)

Messaging: Spring Kafka

Validation: Jakarta Validation

FRONTEND

Framework: Angular 21.0.0

Styling: Angular Material / Bootstrap

State Management: RxJS

DATABASE

- **Primary:** MongoDB (One database per microservice)

DevOps & INFRASTRUCTURE

- **Containerization:** Docker & Docker Compose
- **CI/CD:** Jenkins
- **Quality:** SonarQube
- **Registry:** Netflix Eureka
- **Gateway:** Spring Cloud Gateway

TESTING

- **Unit:** JUnit 5, Mockito
 - **Reactive:** Reactor Test (StepVerifier)
-

6. MICROSERVICES DESIGN

6.1 API Gateway

Responsibilities: Single entry point, Route dispatching, CORS handling.

Routes:

- /auth/** -> auth-service
 - /consumers/** -> consumer-service
 - /utilities/** -> utility-service
 - /readings/** -> meter-service
 - /bills/** -> billing-service
 - /payments/** -> payment-service
-

6.2 Auth Service

Responsibilities: User registration, JWT generation, Role management (Admin/Consumer/Staff).

APIs:

- POST /auth/register
- POST /auth/login
- POST /auth/create-staff (Admin only)

Database: utility_auth_db (users collection)

6.3 Consumer Service

Responsibilities: Managing consumer profiles and linking them to utility connections.

APIs:

- POST /consumers/profile
- GET /consumers/profile/{userId}
- POST /consumers/connections (Request new connection)
- PUT /consumers/connections/{id}/approve (Admin approve)

Database: utility_consumer_db (consumers, connections collections)

6.4 Meter Service

Responsibilities: Recording meter readings and calculating units consumed vs previous month.

APIs:

- POST /readings (Submit reading)
- GET /readings/{meterId} (Get history)

Database: utility_meter_db (readings collection)

6.5 Billing Service

Responsibilities: Orchestrating Bill generation by fetching usage from Meter Service and rates from Utility Service.

- **APIs:**

- POST /bills/generate
- GET /bills/{id}
- GET /bills/consumer/{connectionId}

- **Database:** utility_billing_db (bills collection)
-

6.6 Notification Service

Responsibilities: Listening to Kafka topics and sending emails.

- **Input:** Kafka Topic notification-topic
 - **Logic:** Consumes message -> Sends SMTP email.
-

6.7 Config Server

Responsibilities: Centralized configuration for all microservices.

- **Source:** GitHub Repository (utility-config-server)
- **Profiles:** dev, docker

6.8 Eureka Server

Responsibilities:

- Acts as a **service registry** for all backend microservices.
- Enables **dynamic service discovery**, eliminating hard-coded service URLs.
- Supports **load balancing** and **fault tolerance** by maintaining live service instances.

Behavior:

- All microservices (**Auth, Consumer, Utility, Meter, Billing, Notification**) **register themselves** with the Eureka Server at startup.
- The **API Gateway** queries Eureka to discover available service instances before routing requests.
- Periodic **heartbeat messages** are used to detect unhealthy or unavailable services.

Endpoints:

- Eureka Dashboard: `http://<host>:8761`

7. DATA DESIGN (LLD)

User Document (Auth DB)

```
{
  "id": "u101",
  "username": "john_doe",
  "email": "john@example.com",
  "password": "encrypted_hash",
  "roles": ["ROLE_CONSUMER"],
  "active": true
}
```

Connection Document (Consumer DB)

```
{
  "id": "cn202",
  "consumerId": "p101",
  "utilityType": "ELECTRICITY",
  "meterNumber": "MTR-1001",
  "status": "ACTIVE"
}
```

Tariff Document (Utility DB)

```
{
  "utilityId": "u-elec",
  "name": "Residential Plan A",
  "fixedCharge": 50.0,
  "slabs": [
    { "min": 0, "max": 100, "rate": 5.0 },
    { "min": 101, "max": 500, "rate": 8.0 }
  ]
}
```

Bill Document (Billing DB)

```
{
  "id": "b501",
  "connectionId": "cn202",
  "unitsConsumed": 150.0,
  "amount": 1200.0,
  "status": "UNPAID",
  "billingDate": "2024-01-31"
}
```

8. API DESIGN & VALIDATION

- **RESTful principles:** Resources identified by URIs.
 - **JSON request/response:** Standard format.
 - **Validation:** @Valid annotation on DTOs (e.g., @NotBlank, @Min).
 - **Status Codes:**
 - 200 OK - Success
 - 201 CREATED - Resource created
 - 400 BAD REQUEST - Validation failure
 - 401 UNAUTHORIZED - Invalid JWT
 - 404 NOT FOUND - Resource missing
 - 409 CONFLICT - Duplicate data
-

9. ERROR HANDLING STRATEGY

Global Exception Handling

- Centralized using @RestControllerAdvice.
- Catches WebExchangeBindException (Validation) and ResponseStatusException.

Standard Error Response Format:

```
{
  "timestamp": "2025-01-01T10:00:00",
  "status": 400,
  "error": "Validation Error",
  "message": "Meter reading cannot be negative"
}
```

10. SECURITY DESIGN

Authentication: JWT (JSON Web Tokens) generated by Auth Service.

Authorization: Role-Based Access Control (RBAC) enforced at Gateway and Service layer.

Encryption: BCrypt for password hashing.

Gateway Security: Validates JWT signature before routing.

11. NON-FUNCTIONAL REQUIREMENTS

Area	Design Decision
Scalability	Microservices, Docker, Stateless JWT
Performance	Reactive Programming (WebFlux), Async Messaging (Kafka)
Availability	Service Discovery (Eureka), Independent deployment
Maintainability	Layered Architecture, Centralized Config
Security	RBAC, Input Validation, Encrypted passwords

12. TESTING STRATEGY

Backend

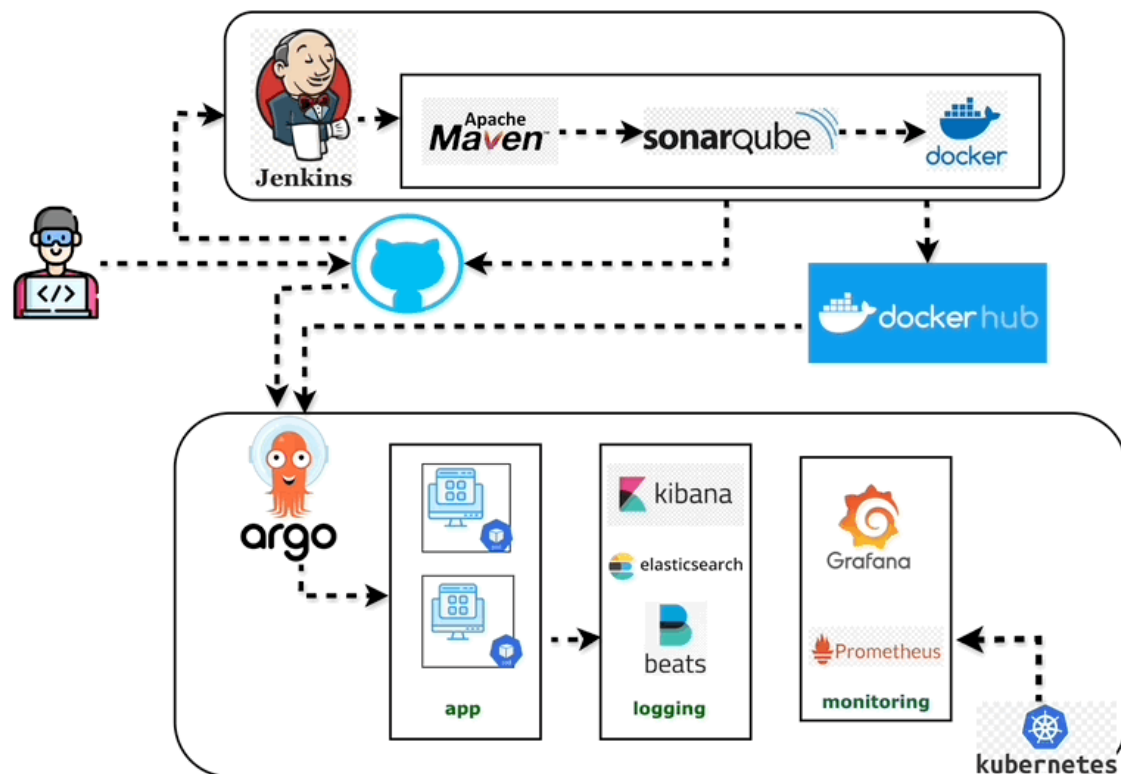
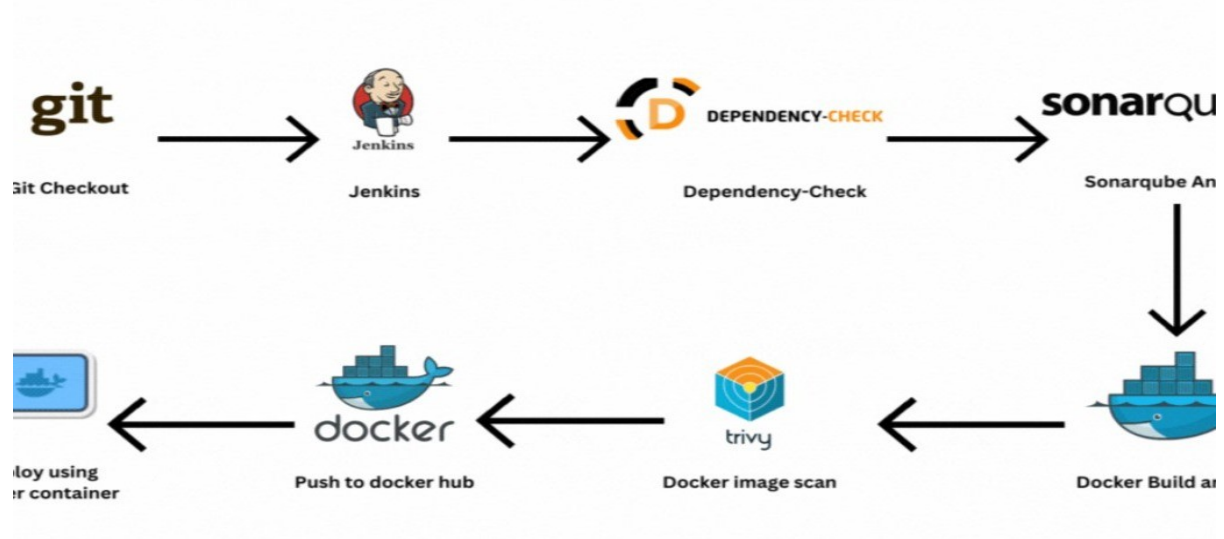
- Unit Tests:** JUnit 5, Mockito (Service layer logic).

- **Integration Tests:** StepVerifier for Reactive flows.
- **Coverage:** 90%.

Quality Gates

- **SonarQube:** Enforced in CI pipeline(>90% coverage).

13. CI/CD DESIGN



Pipeline Flow:

1. **Git Commit** triggers Jenkins.
2. **Build:** Maven Clean Install.
3. **Test:** Run Unit Tests.
4. **Scan:** SonarQube Analysis.
5. **Quality Gate:** Fail if bugs/vulnerabilities found.
6. **Package:** Build Docker Images.
7. **Deploy:** Docker Compose Up.

14. DEPLOYMENT DESIGN

- **Docker:** One container per service.
- **Orchestration:** Docker Compose (for local/dev).
- **Config:** Environment variables injected at runtime.

15. ASSUMPTIONS & CONSTRAINTS

Assumptions

- MongoDB and Kafka are available and running.
- Services communicate over internal Docker network.

Constraints

- No distributed transactions (SAGA pattern out of scope).
- Reactive stack requires learning curve for developers.

16. FUTURE ENHANCEMENTS

- **Payment Gateway Integration:** Credit cards, Debit cards, UPI.
 - **Kubernetes (K8s):** For production orchestration.
 - **ELK Stack:** Centralized logging.
-

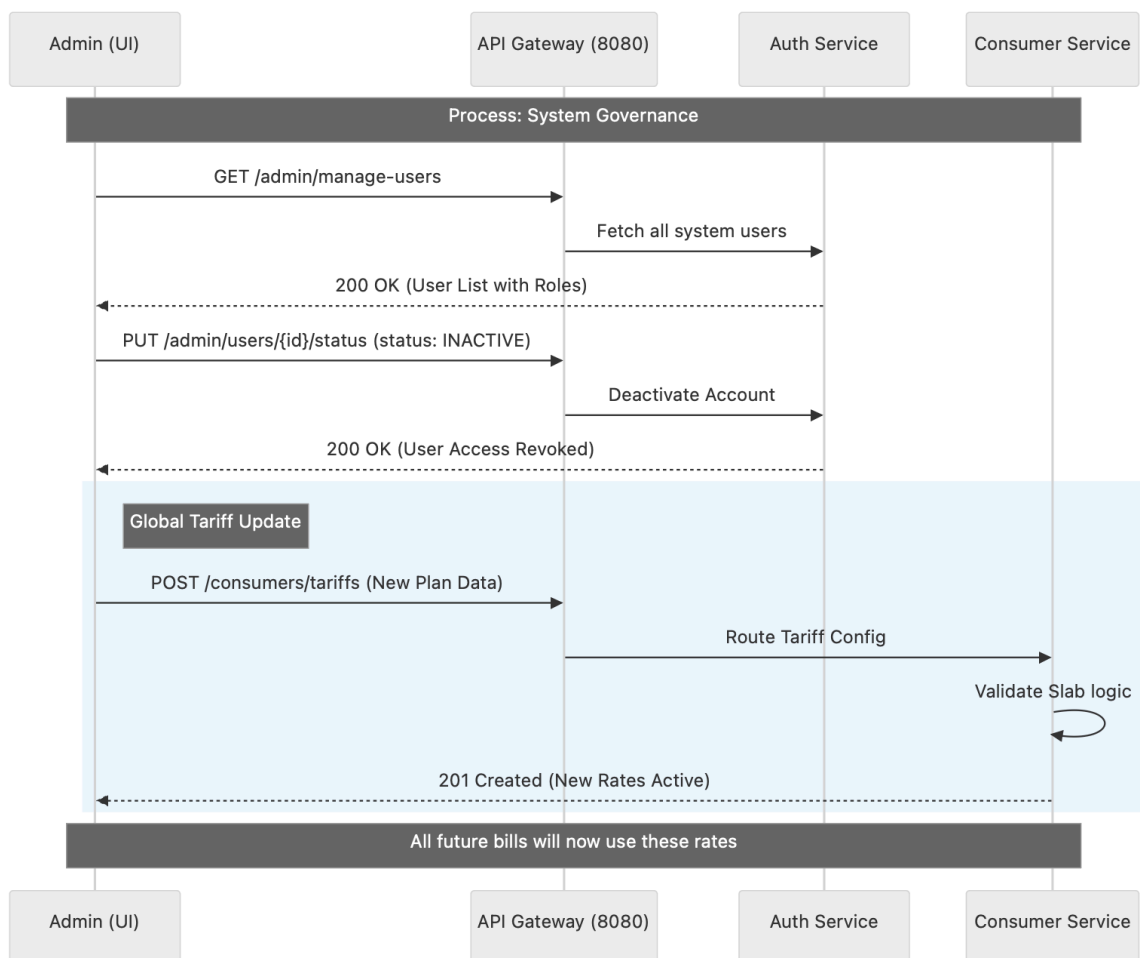
17. CONCLUSION

This design ensures:

- ✓ Clean separation of concerns
- ✓ Scalability via microservices
- ✓ High performance using Reactive streams.
- ✓ Robust error handling and security.

SEQUENCE DIAGRAMS — UTILITY BILLING SYSTEM

1. Admin: System Governance



Scenario

The System Administrator performs high-level governance tasks, including managing user access levels and updating global utility tariff rates.

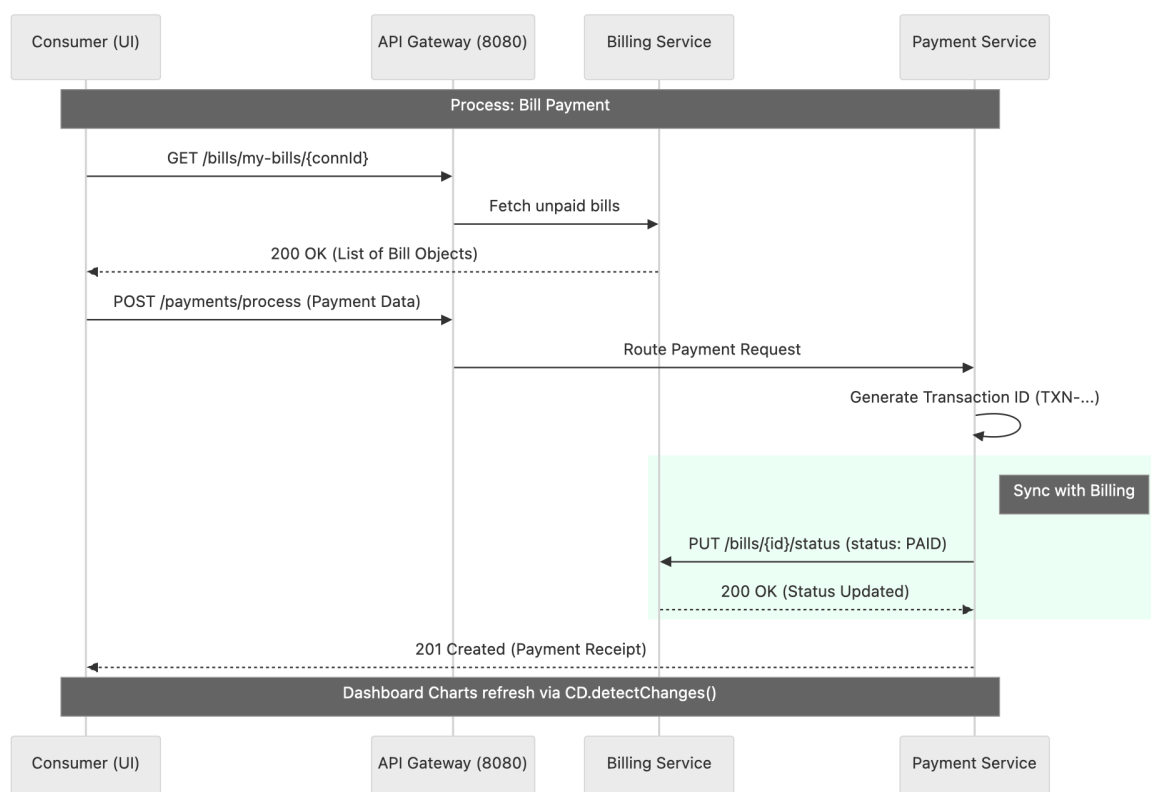
Flow

1. **User Audit:** Admin fetches all system users and their roles from the **Auth Service** via the Gateway.
2. **Access Revocation:** Admin sets a user's status to INACTIVE, and the **Auth Service** revokes their access tokens.
3. **Tariff Update:** Admin submits a new configuration for utility slabs (e.g., new prices for Electricity).
4. **Configuration Logic:** API Gateway routes data to the **Consumer Service**.
5. **Activation:** Consumer Service validates the new slab logic and persists it; all future billing cycles immediately adopt these rates.

Participants

- Admin (UI)
- API Gateway (Port 8080)
- Auth Service
- Consumer Service

2. Consumer: Bill Payment



Scenario

An end-consumer logs into their dashboard to view their outstanding bills and successfully completes a payment.

Flow

1. **Fetch Bills:** Consumer requests their billing history via the UI.
2. **Query:** API Gateway routes the request to the **Billing Service** to fetch all UNPAID records.
3. **Payment Submission:** Consumer selects a bill and submits payment details.
4. **Transaction Logic:** API Gateway routes the request to the **Payment Service**, which generates a unique Transaction ID.
5. **Status Sync:** Payment Service sends a PUT request to the **Billing Service** to update the bill status to PAID.
6. **Receipt Generation:** Payment Service returns a 201 Created response containing the payment receipt.
7. **UI Refresh:** The Consumer UI uses `detectChanges()` to refresh the dashboard charts in real-time.

Participants

- Consumer (UI)
- API Gateway (Port 8080)
- Billing Service
- Payment Service

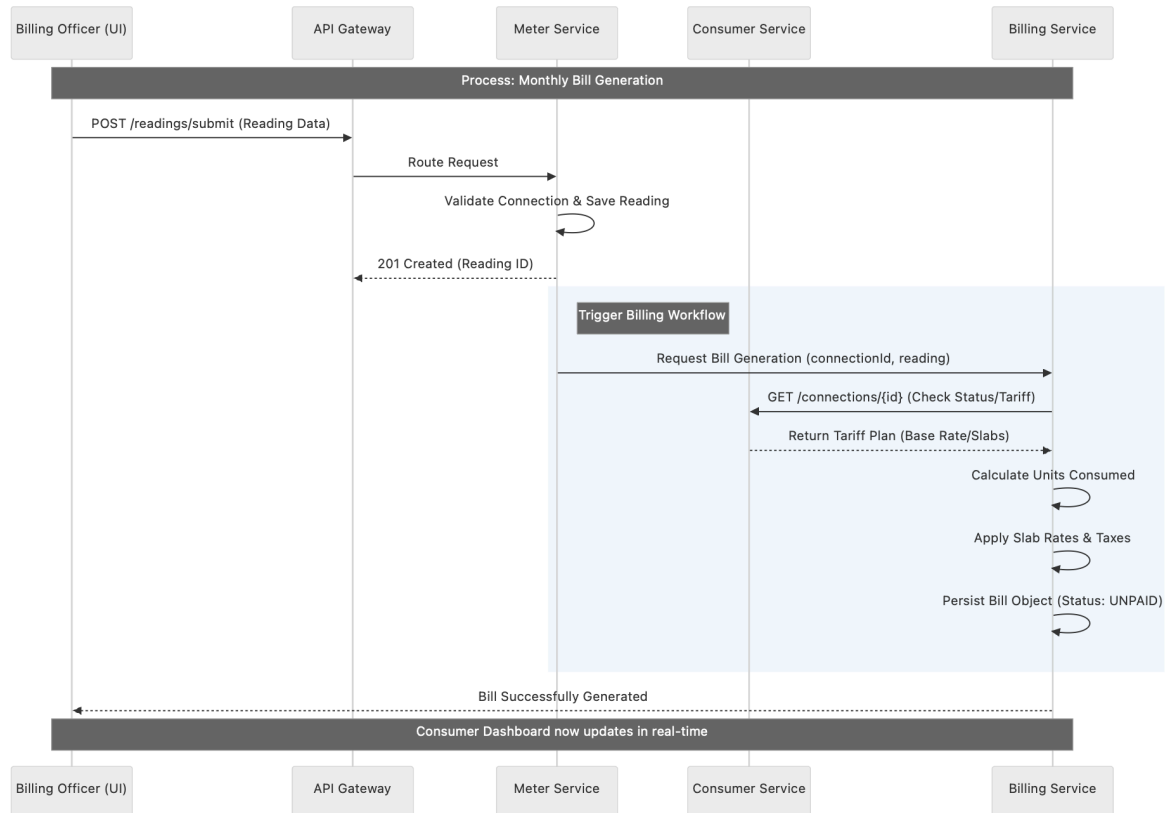
3. Billing Officer: Monthly Bill Generation

Scenario

A Billing Officer submits current meter readings for a consumer to trigger the automated calculation and generation of a monthly utility bill.

Flow

1. **Reading Submission:** Billing Officer submits reading data via the UI.
2. **Routing:** API Gateway routes the POST request to the **Meter Service**.
3. **Validation:** Meter Service validates the consumer connection and persists the reading data.

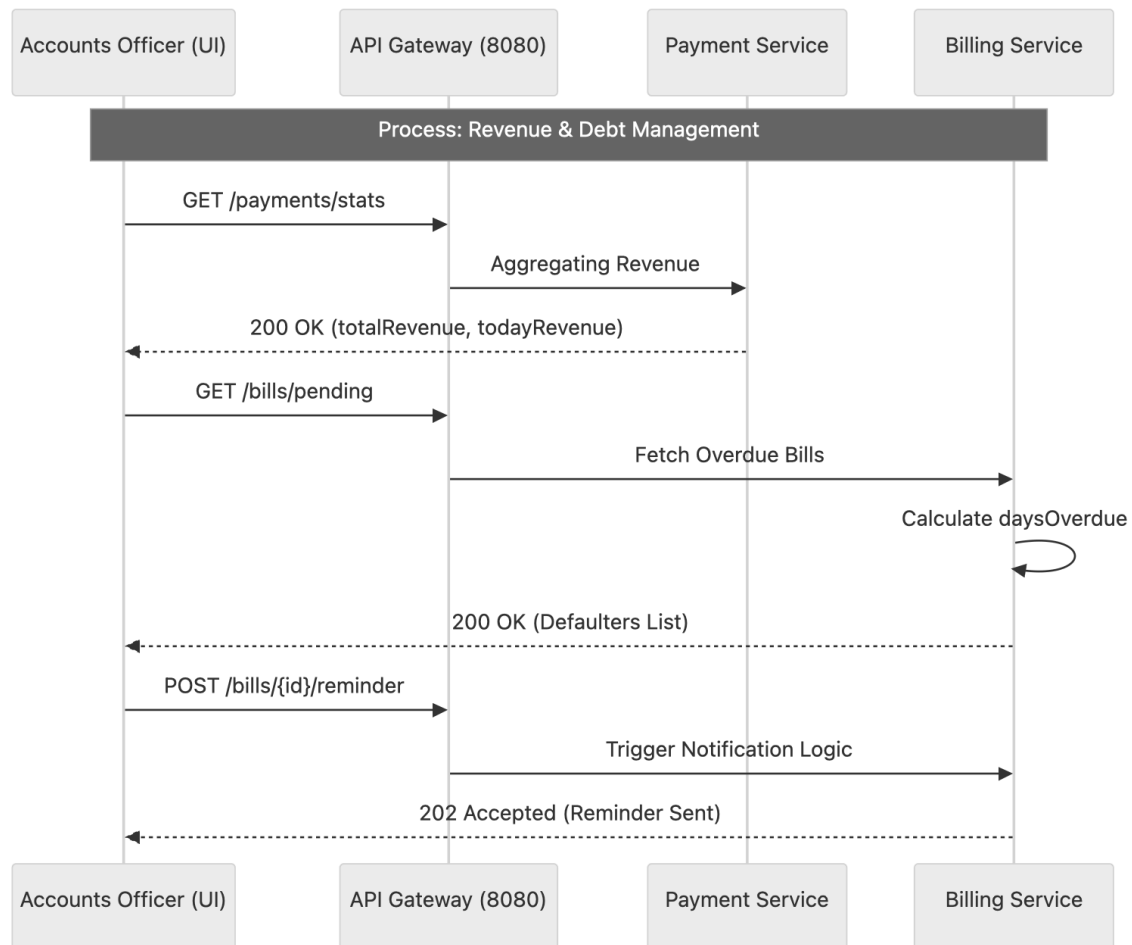


4. **Workflow Trigger:** Meter Service asynchronously triggers the **Billing Service** to generate a bill.
5. **Data Retrieval:** Billing Service fetches the specific tariff plan and consumer status from the **Consumer Service**.
6. **Calculation:** Billing Service calculates units consumed, applies slab-based rates/taxes, and saves the new bill with an UNPAID status.
7. **Confirmation:** A success notification is returned to the Officer.

Participants

- Billing Officer (UI)
- API Gateway
- Meter Service
- Consumer Service
- Billing Service

4. Accounts Officer: Revenue & Debt Management



Scenario

An Accounts Officer monitors the financial health of the utility provider by viewing revenue stats and managing overdue accounts.

Flow

1. **Revenue Aggregation:** Officer requests financial stats; API Gateway fetches aggregated data from the **Payment Service**.
2. **Debt Tracking:** Officer requests the list of pending bills.
3. **Analysis:** **Billing Service** identifies overdue bills, calculates "Days Overdue," and returns a Defaulters List.
4. **Action:** Officer triggers a reminder for a specific bill.
5. **Notification:** API Gateway routes the request to **Billing Service** to execute the notification/reminder logic.

Participants

- Accounts Officer (UI)

- API Gateway (Port 8080)
 - Payment Service
 - Billing Service
-

SEQUENCE → CODE CLASS MAPPING

1) User Registration

Sequence Steps

Sequence Steps:

Sequence Steps: User submits form -> API receives -> Validate -> Encrypt -> Save.

Step	Layer	Class	Responsibility
1	Angular	RegisterComponent	Collect form data
2	API	AuthController	Request mapping
3	Service	AuthServiceImpl	Business logic & Uniqueness check
4	Security	PasswordEncoder	BCrypt encryption
5	Repository	UserRepository	Save user

Key Methods

- UserController.register(UserRegisterRequest)
 - UserService.registerUser(...)
 - UserRepository.existsByEmail(...)
-

2) Bill Generation

Sequence Steps: Admin triggers generation -> Fetch Reading -> Fetch Tariff -> Calculate -> Save.

Code Mapping

Step	Layer	Class	Responsibility
1	API	BillingController	POST /bills/generate
2	Service	BillingServiceImpl	Orchestrate flow
3	Client	WebClient	Call Meter/Utility Services
4	Repo	BillRepository	Save Bill
5	Async	KafkaProducer	Send notification event

CODE SKELETON — PACKAGES & CLASSES

1. AUTH SERVICE (auth-service)

```

com.utility.auth
├── controller
│   └── AuthController.java
├── service
│   ├── AuthService.java (Interface)
│   └── AuthServiceImpl.java (Implementation)
├── repository
│   └── UserRepository.java
├── model
│   └── User.java
├── dto
│   ├── AuthRequest.java
│   └── AuthResponse.java
├── config
│   └── SecurityConfig.java
├── exception
│   └── GlobalExceptionHandler.java

```

Key Responsibilities

- UserController → API layer
- UserService → Business rules
- UserRepository → MongoDB access
- SecurityConfig → Password encoding / JWT (optional)

2. CONSUMER SERVICE (consumer-service)

```
com.utility.consumer
├── controller
│   └── ConsumerController.java
├── service
│   ├── ConsumerService.java (Interface)
│   └── ConsumerServiceImpl.java (Implementation)
├── repository
│   ├── ConsumerRepository.java
│   └── ConnectionRepository.java
├── model
│   ├── Consumer.java
│   └── Connection.java
├── dto
│   └── ConsumerDTO.java
```

Key Responsibilities

- InventoryService → Stock validation & update
- ProductServiceImpl → Core business logic

3. UTILITY SERVICE (utility-service)

```
com.utility.utility
├── controller
│   └── UtilityController.java
├── service
│   ├── UtilityService.java (Interface)
│   └── UtilityServiceImpl.java (Implementation)
├── repository
│   ├── UtilityRepository.java
│   └── TariffRepository.java
├── model
│   ├── Utility.java
│   └── Tariff.java
├── config
│   └── UtilitySeeder.java
```

Key Responsibilities

- ProductClient → Calls Product Service
- OrderStatusValidator → Valid status transitions
- OrderServiceImpl → Orchestrates order flow

4. METER SERVICE (meter-service)

```
com.utility.meter
├── controller
|   ├── MeterController.java
|   └── service
|       ├── MeterService.java (Interface)
|       ├── MeterServiceImpl.java (Implementation)
|       └── repository
|           ├── MeterReadingRepository.java
|           └── model
|               └── MeterReading.java
```

5. BILLING SERVICE (billing-service)

```
com.utility.billing
├── controller
|   ├── BillingController.java
|   └── service
|       ├── BillingService.java (Interface)
|       ├── BillingServiceImpl.java (Implementation)
|       └── repository
|           ├── BillRepository.java
|           └── model
|               ├── Bill.java
|               └── client
|                   ├── WebClientConfig.java (For inter-service calls)
|                   └── dto
|                       ├── MeterReadingDTO.java
|                       └── TariffDTO.java
```

6. NOTIFICATION SERVICE (notification-service)

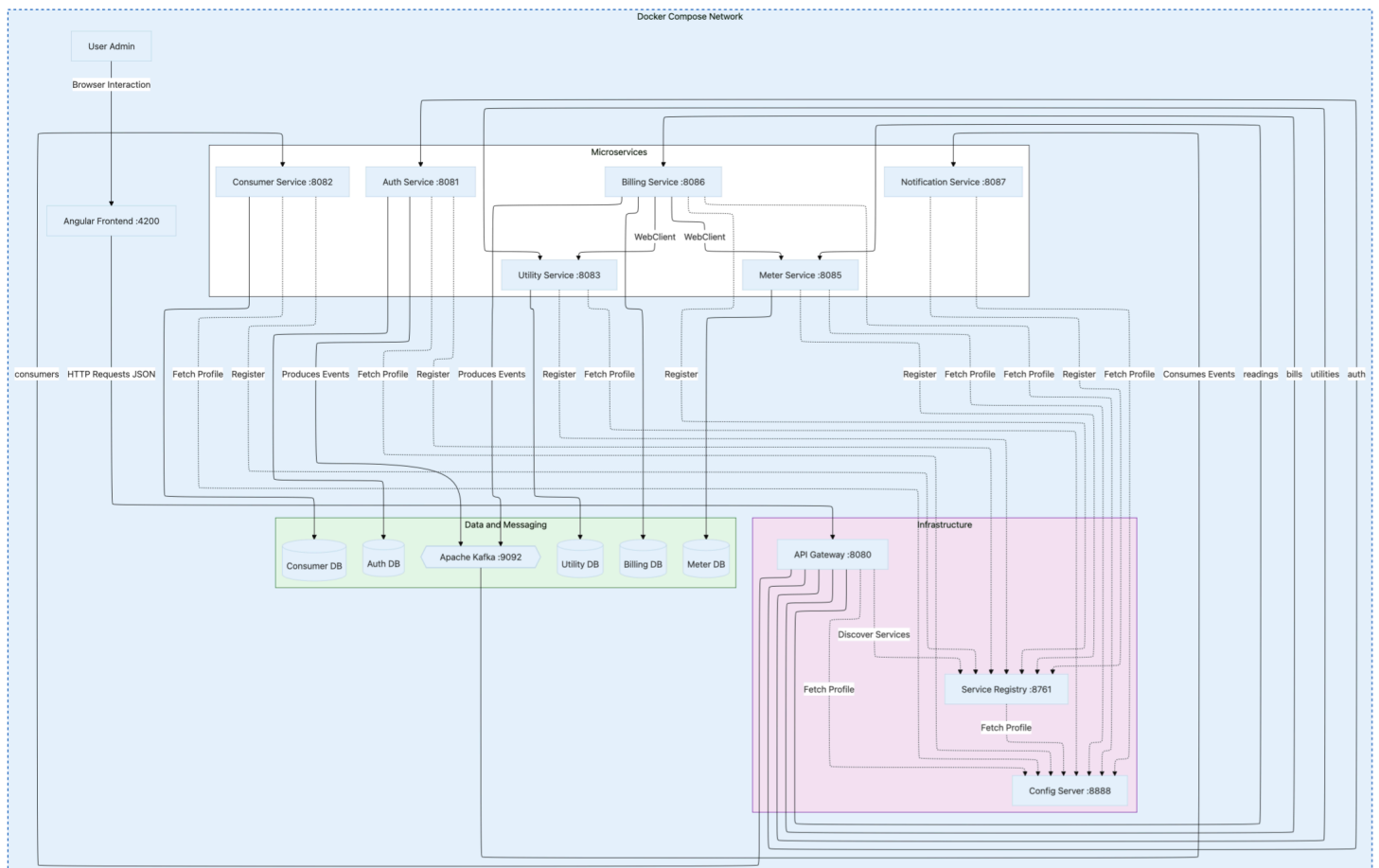
```
com.utility.notification
├── listener
|   ├── NotificationListener.java (Kafka Consumer)
|   └── service
|       ├── EmailService.java (Interface)
|       ├── EmailServiceImpl.java (Implementation)
|       └── dto
|           └── EmailRequest.java
```

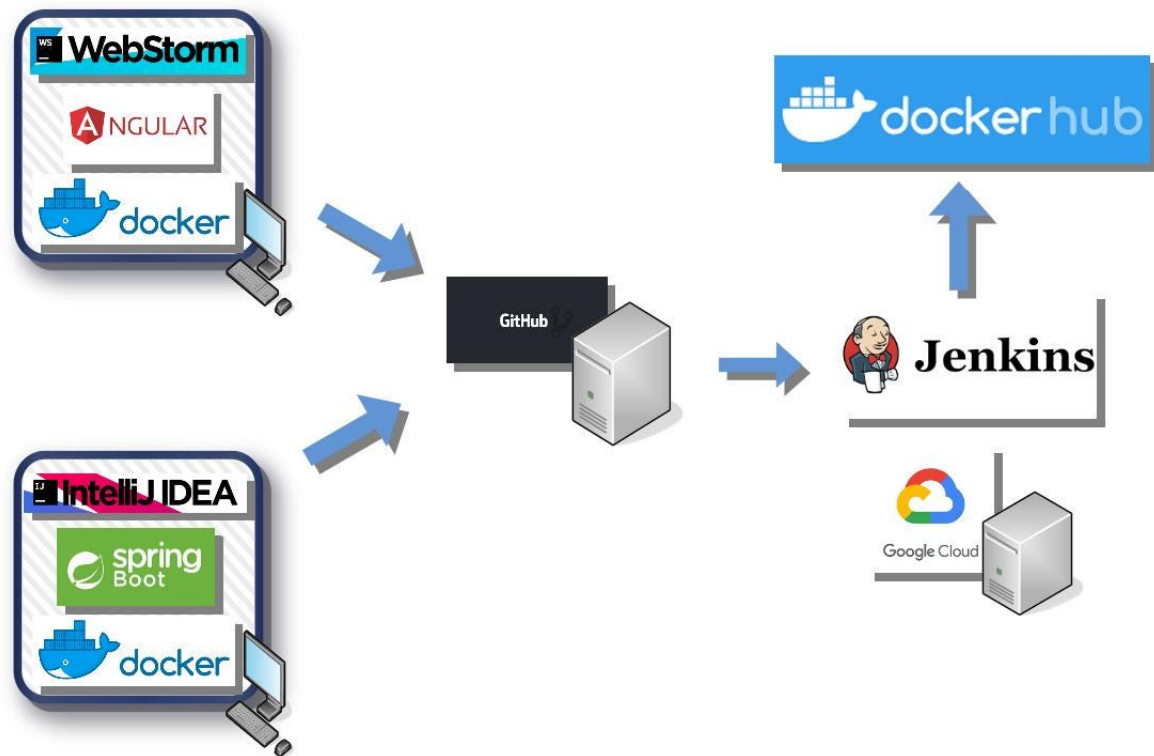
7. REPOSITORY STRUCTURE (ROOT)

```
utility-billing-system/
├── auth-service/
└── consumer-service/
```

- └─ utility-service/
 - └─ meter-service/
 - └─ billing-service/
 - └─ notification-service/
 - └─ api-gateway/
 - └─ discovery-server/
 - └─ config-server/
 - └─ angular-ui/
 - |
 - └─ docker-compose.yml
 - └─ docker-compose-infra.yml
 - └─ README.md
-

Architecture Diagram





BUSINESS RULES

- Only Billing Officers are permitted to assign meter numbers and activate pending connection requests.
- Monthly billing must follow a tiered slab-rate engine that calculates costs based on consumption thresholds and specific tariff categories.
- Bill status must remain 'UNPAID' until the Payment Service generates a unique transaction reference and successfully syncs the data.
- Every utility connection must be uniquely mapped to a single active Meter Number to prevent data overlap and ensure billing accuracy.
- Service access for any consumer account is automatically revoked if the Admin sets the profile status to 'INACTIVE' in the Auth Service.